

← Transmitting and Decoding APRS Packets with ADALM PlutoSDR on Mac OS using PythonDMR and Pi-Star →

Transmitting and Decoding APRS Packets with ADALM PlutoSDR on Raspberry Pi using Python

BY AP, MAY 4, 2026

This post is a Raspberry Pi 5 edition of our AFSK APRS transmitter and decoder project using the ADALM PlutoSDR. We cover everything from scratch: setting up the operating system, installing all dependencies including libiio, SoapySDR and pyadi-iio, connecting the PlutoSDR over USB, and running the full Python transmitter and decoder pipeline.

What is APRS?

APRS (Automatic Packet Reporting System) is a digital communications protocol used in amateur radio. It transmits short packets containing position reports, messages, weather data, and more. It uses **AFSK modulation** (Audio Frequency Shift Keying) at 1200 baud, with:

- **Mark frequency:** 1200 Hz (represents bit 1)
- **Space frequency:** 2200 Hz (represents bit 0)
- **Carrier frequency:** 144.800 MHz in Europe, 144.390 MHz in North America

The underlying framing protocol is **AX.25**, which handles callsigns, addressing, CRC error checking, NRZI encoding, and bit stuffing.

Hardware Requirements

- Raspberry Pi 5 (4 GB or 8 GB recommended)
- MicroSD card, 16 GB or larger, with Raspberry Pi OS (64-bit, Trixie)
- Better, you might consider a NVMe unit via PCIe (Raspberry Pi 5 supports that)
- ADALM PlutoSDR
- A **data-capable** USB cable (not a charge-only cable)
- A 2-meter amateur radio antenna connected to the PlutoSDR TX port
- An amateur radio licence to transmit on 144 MHz
- Internet connection for package installation

Related posts

Setting Up SDR Tools on Raspberry Pi: A Complete Installation Guide
BY AP, JUNE 15, 2026
Raspberry Pi | Raspberry Pi 5 | sdr | SoapySDR

Getting Software Defined Radio (SDR) hardware to work on a Raspberry Pi can be a frustrating experience. Between conflicting library versions, missing dependencies, and the need to compile several components from source, what should be a simple setup often turns into a multi-hour debugging session. To make this process repeatable and reliable, I put together […]

The Complete Amateur Radio SDR Buyer's Guide and Comparison (2025)
BY AP, JUNE 1, 2026
adalm pluto | sdr

Software Defined Radio (SDR) has transformed the radio hobby. What once required racks of analog hardware can now be done with a small USB dongle, a laptop, and open-source software. But as the market has matured, the choices have multiplied — from sub-$30 dongles to $1,000+ research-grade platforms. This guide covers every major SDR available […]

Part 1: Raspberry Pi OS Setup

Flash the OS

Download and install the Raspberry Pi Imager from raspberrypi.com. Flash **Raspberry Pi OS Lite 64-bit** or **Raspberry Pi OS Desktop 64-bit** to your microSD card. Enable SSH and set your username and password in the imager's advanced settings before flashing.

First Boot and System Update

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

Install Essential Build Tools

```
sudo apt install -y \
    build-essential \
    cmake \
    git \
    wget \
    curl \
    pkg-config \
    libusb-1.0-0-dev \
    libxml2-dev \
    libavahi-client-dev \
    bison \
    flex \
    python3 \
    python3-pip \
    python3-venv \
    python3-dev \
    libpython3-dev \
    usbutils \
    net-tools \
    iputils-ping
```

Part 2: Install libiio (Analog Devices IIO Library)

`libiio` is the low-level C library that talks directly to the PlutoSDR hardware. It must be built from source on the Raspberry Pi to get the latest version.

Clone and Build libiio

```
cd ~
git clone https://github.com/analogdevicesinc/libiio.git
cd libiio
mkdir build && cd build

cmake \
    -DCMAKE_BUILD_TYPE=Release \
    -DPYTHON_BINDINGS=ON \
    -DWITH_TESTS=ON \
    -DWITH_EXAMPLES=ON \
    -DENABLE_DNS_SD=ON \
    ..

make -j4
sudo make install
sudo ldconfig
```

Verify libiio Installation

```
iio_info -version
```

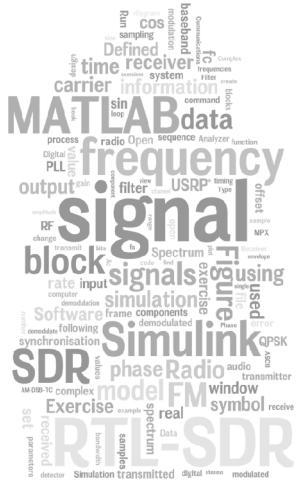
You should see output similar to:

```
libiio version: 0.25 (git tag: v0.25)
Compiled with backends: local xml ip usb serial
```

Part 3: Install libad9361 (PlutoSDR Filter Library)

This library provides the AD9361 RF chip calibration and filter routines used by the PlutoSDR.

```
cd ~
git clone https://github.com/analogdevicesinc/libad9361-iio.git
cd libad9361-iio
mkdir build && cd build
```



Transmitting and Decoding APRS Packets with ADALM PlutoSDR on Mac OS using Python

BY AP, MAY 11, 2026

adalm pluto | APRS | matlab

This post documents an experimental, complete, journey of building an AFSK APRS packet transmitter and receiver using the ADALM PlutoSDR software-defined radio, first in MATLAB and then in Python. I tried to cover everything from generating AX.25 frames, AFSK modulation, hardware connection troubleshooting on macOS, and finally a full packet decoder. What is APRS? APRS […]



RPi5 Boot Error Log

BY AP, APRIL 11, 2026

journalctl | kernel | Raspberry Pi | Raspberry Pi 5

With the idle freeze fixed and persistent journal logging in place, I ran the previous boot error log to see what remained: Six errors, four distinct sources. Here is what each one means and what we did about them. Error 1: alsa-restore.rules — Missing Label What is it? When a sound card is detected, udev […]

```
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j4
sudo make install
sudo ldconfig
```

Part 4: Install SoapySDR

`SoapySDR` is a vendor-neutral SDR hardware abstraction layer. It allows many different SDR applications to talk to many different hardware devices through a single unified API. Installing it makes the PlutoSDR accessible to GNU Radio, SDR++, CubicSDR, and any other SoapySDR-compatible application.

Install SoapySDR Core

```
cd ~
git clone https://github.com/pothosware/SoapySDR.git
cd SoapySDR
mkdir build && cd build

cmake \
    -DCMAKE_BUILD_TYPE=Release \
    -DPYTHON3_EXECUTABLE=$(which python3) \
    ..

make -j4
sudo make install
sudo ldconfig
```

Verify SoapySDR

```
SoapySDRUtil --info

######################################################
##     Soapy SDR -- the SDR abstraction library     ##
######################################################

Lib Version: v0.8.1-g19a8a9b6
API Version: v0.8.0
ABI Version: v0.8
Install root: /usr/local
Search path:  /usr/local/lib/SoapySDR/modules0.8
No modules found!
```

No modules found is expected at this stage. The PlutoSDR module is installed next.

Install SoapyPlutoSDR Module

```
cd ~
git clone https://github.com/pothosware/SoapyPlutoSDR.git
cd SoapyPlutoSDR
mkdir build && cd build

cmake \
    -DCMAKE_BUILD_TYPE=Release \
    ..

make -j4
sudo make install
sudo ldconfig
```

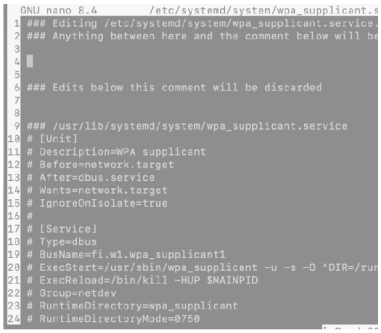
Verify PlutoSDR is Found by SoapySDR

Plug in the PlutoSDR via USB, wait 10 seconds, then run:

```
SoapySDRUtil --find="driver=plutosdr"

######################################################
##     Soapy SDR -- the SDR abstraction library     ##
######################################################

Found device 0
  addr = ip:192.168.2.1
  driver = plutosdr
  label = PlutoSDR USB [ip:192.168.2.1]
  media = USB 2.0
  module = SoapyPlutoSDR
  serial = 10447355…
```

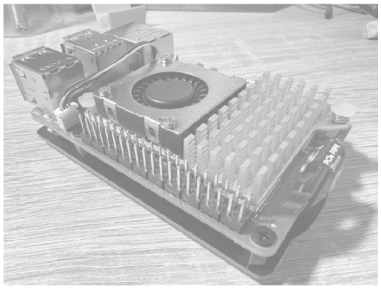


Debugging a Raspberry Pi 5 Idle Freeze

BY AP, APRIL 10, 2026

claude ai | journalctl | NVMe | Raspberry Pi | Raspberry Pi 5

Lately I am extensively using Claude from Anthropic for my hobbies and work. This is a live, interactive, debugging session documenting how we (me and Claude) diagnosed and fixed an unexplained system freeze on a Raspberry Pi 5 running Raspberry Pi OS, booting from an NVMe SSD over PCIe.

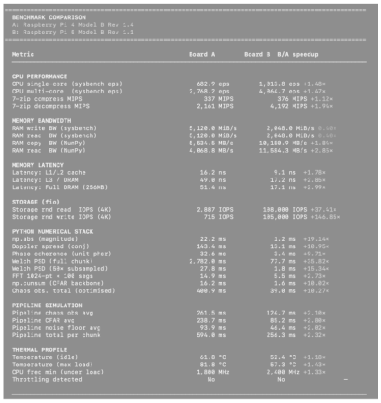


What Actually Goes Wrong with NVMe

BY AP, APRIL 8, 2026

NVMe | Raspberry Pi | Raspberry Pi 5

This post documents a complete troubleshooting session getting a Raspberry Pi 5 to boot from a Samsung NVMe SSD running a freshly written Debian Trixie image.



Adalm Pluto ? Don't think so

BY AP, NOVEMBER 11, 2025

adalm pluto | Linux | sdr

Part 5: Connect the PlutoSDR on Raspberry Pi

Verify USB Detection

```
lsusb
```

Look for a line like:

```
Bus 001 Device 003: ID 0456:b673 Analog Devices, Inc. PlutoSDR
```

Check the USB Network Interface

The PlutoSDR creates a virtual USB network adapter using the CDC-ECM driver, which Raspberry Pi OS supports natively. No driver installation is needed.

```
ip addr show
```

Look for an interface (typically *usb0* or *enx**) with the address 192.168.2.10 assigned automatically, or assign it manually:

```
# Check which interface appeared after plugging in
ip link show

# If no IP was assigned automatically, set one manually
sudo ip addr add 192.168.2.10/24 dev usb0
sudo ip link set usb0 up

# Verify
ping -c 4 192.168.2.1
```

```
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=0.452 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=0.398 ms
```

Make the IP Assignment Permanent

To avoid having to reassign the IP every reboot, create a systemd-networkd config file:

```
sudo nano /etc/systemd/network/pluto.network
```

Add the following content:

```
[Match]
MACAddress=00:e0:22:fe:da:c9

[Network]
Address=192.168.2.10/24
```

Replace the MAC address with the one shown by *ip link show* for your PlutoSDR interface. Then enable networkd:

```
sudo systemctl enable systemd-networkd
sudo systemctl restart systemd-networkd
```

Alternatively, add a udev rule to always name the interface *pluto0*:

```
sudo nano /etc/udev/rules.d/99-plutosdr.rules
```

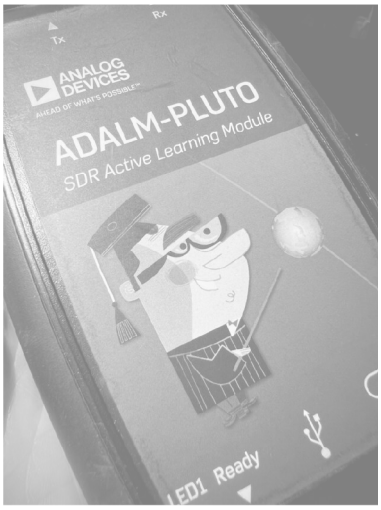
```
SUBSYSTEM==”net”, ATTRS{idVendor}==”0456”, ATTRS{idProduct}==”b673”, NAME=”pluto0”
```

```
sudo udevadm control –reload-rules
sudo udevadm trigger
```

Test libiio Connection to PlutoSDR

```
iio_info -u ip:192.168.2.1
```

I have a love-hate relationship with Adalm Pluto. I bought it from Mouser in 2020 when it was roughly 100 euros (now it is almost double that price !) and I never managed to do anything useful with it.



My first order from RF Parts…

BY AP, MAY 13, 2014

144.8 | APRS | Mitsubishi | MOSFET | RA08H1317M | RA30H1317M

I was looking for a RF power module for my APRS beacon project and stumbled upon RF Parts. They are somehow of a niche−oriented online electronic part retailers, specialised on RF and — it seems — mostly on Ham Radio RF parts. I placed my order for two Mitsubishi power RF MOSFET Amplifier Modules and the package […]



LTSpice Parameter Sweep

BY AP, APRIL 14, 2014

.param | .step | APRS | Bode plot | filters | LTSpice | LTSpice IV | SPICE

While designing/ working on my APRS project, I used LTSpice IV for some simulations. For those that don’t know, LTspice IV is a high performance SPICE simulator, schematic capture and waveform viewer from Linear Technology. It is a freeware and can be downloaded from this link. Versions for my both platforms exist, Win 7.0 and Mac. […]

You should see several pages of output listing all IIO devices and channels on the AD9363 chip. If this works, the hardware layer is fully operational.

Part 6: Set Up Python Virtual Environment

Always use a virtual environment on Raspberry Pi OS Trixie to avoid conflicts with system Python packages.

```
cd ~
mkdir aprs_pluto && cd aprs_pluto
python3 -m venv venv
source venv/bin/activate
```

Install Python Dependencies

```
pip install –upgrade pip
pip install numpy scipy
```

Install pyadi-iio

`pyadi-iio` is Analog Devices' official Python interface for the PlutoSDR. It wraps `libiio` with a clean, high-level API.

```
pip install pyadi-iio
```

If pip installation fails, build from source:

```
cd ~
git clone https://github.com/analogdevicesinc/pyadi-iio.git
cd pyadi-iio
pip install .
cd ~/aprs_pluto
source venv/bin/activate
```

Install SoapySDR Python Bindings

```
pip install SoapySDR
```

If the pip package does not match your compiled SoapySDR version, install the Python bindings from the source build instead:

```
cd ~/SoapySDR/build
sudo make install
# The Python bindings are installed system-wide by cmake.
# Copy them into the venv site-packages:
cp -r /usr/local/lib/python3.*/dist-packages/SoapySDR* \
      ~/aprs_pluto/venv/lib/python3.*/site-packages/
```

Quick Connection Test

show source

```
python3 test_pluto.py

Connected via ip:192.168.2.1
   Sample rate:  575999 Hz
   TX LO:        2399999998 Hz
   RX LO:        2399999998 Hz
```

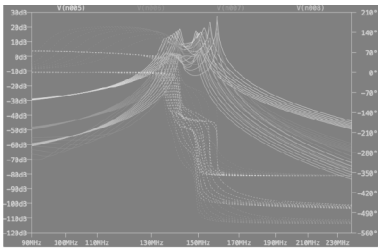
The small frequency rounding differences (575,999 instead of 576,000, etc.) are normal. The AD9363 PLL cannot hit exact values and rounds to the nearest achievable clock. This has no practical effect on APRS operation.

Part 7: Optional — Install GNU Radio with PlutoSDR Support

If you want a graphical signal analyser and flowgraph environment alongside the Python scripts, install GNU Radio and its PlutoSDR sink and source blocks.

```
sudo apt install -y gnuradio
pip install gnuradio-companion

# Install gr-iio (PlutoSDR blocks for GNU Radio)
cd ~
```

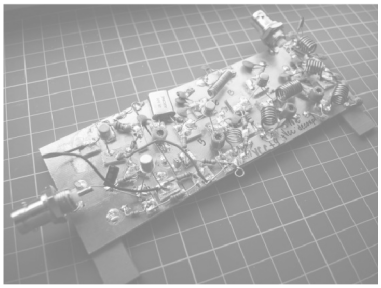


144.8 Mhz TX for APRS (3)

BY AP, APRIL 11, 2014

144.8 | 2m | APRS | Colpitts | DSOX 2002A | quartz | transmitter | TX

This is a quick update post. The road I took with this little project is slowly getting to a dead end due to what it seems to be a lack of available crystals for 144.8 (like 48.26 or 28.96 MHz, for using the third or fifth harmonic). I believe that, currently, all APRS modules are […]



144.8 Mhz TX for APRS (2)

BY AP, APRIL 10, 2014

144.8 | 2m | APRS | Colpitts | DSOX | DSOX 2002A | FFT | LTSpice | LTSpice IV | quartz | SPICE | transmitter | TX

More work today and yesterday on this project. Still locking on 144 MHz, but this is normal, the crystal I used is a 48 MHz overtoned, working on the third harmonic in this case, thus spot on 144.0 MHz: I managed to clean the output, at the expense of transmitted power. I used a 7-pole 2m bandpass […]



144.8 Mhz TX for APRS

BY AP, APRIL 4, 2014

144.8 | APRS | beacon | DSOX 2002A | FFT | LTSpice | LTSpice IV | Mask Limit Testing | quartz | SPICE

Been busy at work lately, but still had some time for myself. Latest idea: a low-cost APRS beacon, done with a classical 144.8 Mhz TX and a µController. AX.25 on air. Automatically.

```
git clone https://github.com/analogdevicesinc/gr-iio.git
cd gr-iio
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j4
sudo make install
sudo ldconfig
```

Part 8: APRS Transmitter

How the Transmitter Works

- Build an AX.25 UI frame with callsign encoding, CRC-16, and bit stuffing
- Apply NRZI encoding
- Modulate using AFSK at 48 kHz internally
- Upsample to 576 kHz to satisfy the pyadi-iio minimum sample rate of 521 kHz
- Transmit as complex IQ via the PlutoSDR at 144.800 MHz

Transmitter Code

show source

Run the Transmitter

```
cd ~/aprs_pluto
source venv/bin/activate
python3 aprs_tx.py

=== AFSK APRS Transmitter – ADALM PlutoSDR / Raspberry Pi 5 ===

Building AX.25 frame…
  312 bits after stuffing
Modulating AFSK…
  Duration at 48000 Hz: 3.21 s
  Resampling 48000 Hz -> 576000 Hz  (P=12, Q=1)
  Final samples at 576000 Hz: 18489600 (3.21 s)

Searching for PlutoSDR…
  Trying ip:192.168.2.1 … Connected!

Configuring TX on 144.800 MHz…
  Sample rate: 575999 Hz
  TX LO:       144799998 Hz
  TX gain:     -10 dB

Transmitting on 144.800 MHz…
  Cycling for 9.6 s (3 passes)…

Done.
```

Part 9: APRS Decoder

How the Decoder Works

- Capture IQ at 576 kHz from the PlutoSDR RX port
- Downsample to 48 kHz
- FM discriminator to extract instantaneous frequency
- Bandpass filter from 1000 to 2400 Hz
- Dual-tone energy detection comparing 1200 Hz and 2200 Hz envelopes via Hilbert transform
- Bit clock recovery using zero-crossing synchronisation
- NRZI decode and bit destuffing
- AX.25 flag boundary search
- CRC-16 verification
- AX.25 frame decode including callsign, SSID and digipeater path
- APRS info field parsing for position, message, status, weather and object report types

Decoder Code

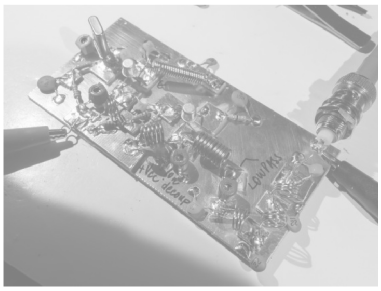
show source

Run the Decoder

```
cd ~/aprs_pluto
source venv/bin/activate
python3 aprs_rx.py

=== AFSK APRS Decoder – ADALM PlutoSDR / Raspberry Pi 5 ===
```

So these days I've been busy making this: Winding coils, soldering, measuring. Fun. The frequency is remarkable stable, given the rudimentary nature of this. For example, […]



```
    Frequency:   144.800 MHz
    Sample rate: 576 kHz
    Baud rate:   1200 bps
    Tones:       1200/2200 Hz

Connected
  RX LO:        144.800 MHz
  Sample rate:  575999 Hz

Listening for APRS packets… (Ctrl+C to stop)

============================================================
  2024-03-24 14:32:07
  NOCALL -> APRS
  Raw info: !4807.38N/01131.00E>Test APRS via PlutoSDR
  Position:  48.12300, 11.51667
  Comment:   Test APRS via PlutoSDR
============================================================
  Total packets decoded: 1
```

Part 10: Run as a systemd Service

To run the decoder automatically at boot as a background service:

```
sudo nano /etc/systemd/system/aprs-decoder.service
```

This returns:

```
[Unit]
Description=AFSK APRS Decoder via PlutoSDR
After=network.target

[Service]
ExecStart=/home/pi/aprs_pluto/venv/bin/python3 /home/pi/aprs_pluto/aprs_rx.py
WorkingDirectory=/home/pi/aprs_pluto
StandardOutput=journal
StandardError=journal
Restart=always
RestartSec=5
User=pi

[Install]
WantedBy=multi-user.target
```

Then run:

```
sudo systemctl daemon-reload
sudo systemctl enable aprs-decoder
sudo systemctl start aprs-decoder
```

Watch live output

```
journalctl -u aprs-decoder -f
```

Troubleshooting on Raspberry Pi

- **lsusb does not show the PlutoSDR:** The USB cable is charge-only. Replace it with a data cable. Test with a different USB port on the Pi.
- **No `usb0` or `enx` interface after plugging in:** Run `dmesg | grep -i cdc` to check whether the CDC-ECM driver loaded. On rare occasions you may need to load it manually with `sudo modprobe cdc_ether`.
- **ping 192.168.2.1 fails even though interface exists:** Assign the IP manually with `sudo ip addr add 192.168.2.10/24 dev usb0` and bring the interface up with `sudo ip link set usb0 up`.
- **`iio_info` hangs or times out:** The PlutoSDR firmware may be too old. SSH into the Pluto with `ssh root@192.168.2.1` (password: analog) and run `version` to check. Update the firmware via the Analog Devices wiki if needed.
- **`pyadi-iio` sample rate error below 521e3:** Always use 576,000 Hz or higher. Never pass 48,000 or 96,000 Hz directly to the PlutoSDR via `pyadi-iio`.
- **TX cyclic buffer exception:** With `tx_cyclic_buffer = True`, call `sdr.tx()` only once. Use `time.sleep()` to control duration, then call `sdr.tx_destroy_buffer()`.
- **Decoder runs but never finds packets:** Check RX gain. Try setting `sdr.gain_control_mode_chan0 = 'manual'` and `sdr.rx_hardwaregain_chan0 = 40` for stronger amplification. Also verify the

antenna is connected to the RX port, not the TX port.

Key Lessons

- Always use a **data-capable USB cable**. This is the most common source of connection failures on any platform.
- The PlutoSDR communicates with pyadi-iio entirely over **IP at 192.168.2.1** via the USB virtual network adapter. The serial port is only for shell access.
- Choose a sample rate that is an **exact integer multiple of 48,000 Hz** and above 521,000 Hz. 576,000 Hz (12 times 48 kHz) is the ideal choice.
- Build **libiio from source** on Raspberry Pi. The apt package is often several versions behind and may have IIO context bugs with newer PlutoSDR firmware.
- **SoapySDR** is not strictly required for the Python scripts but is strongly recommended if you also want to use GNU Radio, SDR++ or any other graphical SDR tool with the same hardware.
- The AD9363 PLL rounds frequencies slightly. A 2 Hz error on 144.800 MHz is 0.0000014% and has no effect on APRS reception or transmission.

CATEGORIES: HAM RADIO, PROGRAMMING, PYTHON • TAGS: ADALM PLUTO, AFSK, APRS, AX.25, LIBIIO, PYADI-IIO, RASPBERRY PI 5

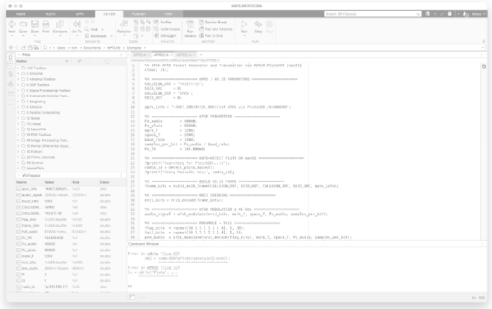
Previous and next posts

← Transmitting and Decoding APRS Packets with ADALM PlutoSDR on Mac OS using PythonDMR and Pi-Star →

Transmitting and Decoding APRS Packets with ADALM PlutoSDR on Mac OS using Python

BY AP, MAY 11, 2026

This post documents an experimental, complete, journey of building an AFSK APRS packet transmitter and receiver using the ADALM PlutoSDR software-defined radio, first in MATLAB and then in Python. I tried to cover everything from generating AX.25 frames, AFSK modulation, hardware connection troubleshooting on macOS, and finally a full packet decoder. What is APRS? APRS […]



DMR and Pi-Star

BY AP, APRIL 27, 2026

Digital Mobile Radio (DMR) has become one of the most popular digital voice modes in amateur radio, offering efficient use of spectrum, clear audio, and a global network of repeaters and hotspots. Combined with Pi-Star — a purpose-built operating system for MMDVM-based hotspots — even operators far from a DMR repeater can enjoy worldwide digital […]

Comments are closed.

Search Alauda Projects... Search

3D Print

Apple Programming

Arduino

Blog

Books

C projects

Coding for web

Contests

Debugging
Electronics
Equipment
Field Day
FPGA
Fusion 360
Ham Radio
HDL
HiFi
How To's
Kernel and drivers
Linux and related
Measurement & Analysis
Microcontrollers
Other
PCB Development
Programming
Projects
Python
Reviews
Simulations
Tutorials
Woodworking
Wordpress